Homework 1 – Version C: Air Quality in Tuzla Canton

Course: Mobile Application and Service Development **Level:** advanced · **Points:** 7 + 1 bonus activity point · **Deadline:** 31.10.2026 at 23:59 · **Defense:** lab session, 02.11.2026

Description

The repository contains real daily air quality data for two monitoring stations, Skver (Tuzla) and Lukavac, from December 2016 to October 2026. The data is not clean: the lines are not sorted by date, and on many days the sensors measured nothing. You write a Kotlin program that reads both files, separates valid lines from invalid ones and prints a report comparing the two stations. You design the data model yourself. At the defense you will justify your model.

Data source: World Air Quality Index Project (aqicn.org), based on measurements by the Federal Hydrometeorological Institute and the Ministry of Spatial Planning and Environmental Protection of Tuzla Canton.

Input data

The files are `data/skver.csv` and `data/lukavac.csv`. Both have the same format:

```
date, pm25, pm10, o3, no2, so2, co
2026/10/1, 50, 24, 38, 15, 8,
2026/10/9, 47, 24, 13, 11, ,
```

- The first line is a header.
- Fields are separated by commas, and there may be a space after a comma.
- The date is in the format `year/month/day`, without leading zeros.
- The values are **AQI indices** (whole numbers), not concentrations.
- An empty field means there was no measurement that day.

For this homework you only need the **date** and **PM2.5**, but a line is valid only if it has all 7 fields.

Read a file like this (add `import java.io.File` at the top of the file):

kotlin

```kotlin
val lines = File("data/skver.csv").readLines()
```

AQI categories

PM2.5 AQI	Category
0–50	Good
51–100	Moderate
101–150	Unhealthy for Sensitive Groups
151–200	Unhealthy
201–300	Very Unhealthy
above 300	Hazardous

Model requirements

Your model must satisfy the following:

1. Two daily measurements with the same data must be equal when compared with `==`, and printing a measurement with `println` must be readable, without writing that logic by hand.
2. Measurement data cannot be changed after the measurement is created.
3. A day without a measurement has no value. If you try to calculate with a value (e.g. add it up or compute an average) without checking that it exists, the program must not compile.
4. Dates must be compared and sorted correctly. Note: as text, `"2026/10/1"` comes before `"2026/9/30"`, which is wrong.
5. AQI categories are a fixed set. A misspelled category must not compile, and the range limits belong to the category instead of being scattered around the code.
6. The result of reading one line is **either** a measurement **or** an error that carries the line number and the reason. An error must not have measurement fields, and a measurement must not have error fields.
7. If a new kind of reading result is added later, the compiler itself must show every place in the code that needs to handle it.

Reading the text

- Write a function that creates a reading result from **one line** of text and its line number. Lines are numbered as in the file: the header is line 1, the first data line is line 2.
- The program **must keep running** on every line.
- Spaces around commas must not cause problems.
- Print the error reason with exactly these words:
  - `wrong number of fields`

    - `invalid date: <text>`
    - `value is not a number: <text>`
- All lines in these files are valid, but at the defense you will get invalid lines too. If there are any, print them below the "Invalid lines" line, each in the format `  Line <number>: <reason>`.

What the program must calculate

For **each station**:

1. How many days were loaded, how many lines are invalid and how many days have no PM2.5.
2. The measurement period, from the earliest to the latest date.
3. The worst day, with the highest PM2.5. If there are several, take the earliest.
4. The number of days per AQI category, in the order of the table. Days without PM2.5 are not counted.
5. The most polluted and the cleanest month, by average PM2.5 across all years.

For **both stations together**:

6. The average PM2.5 per year, for both stations on the same line, from the earliest to the latest year. If a station has no data for a year, print `-` instead of the average (an empty field in CSV format).
7. Print the same yearly averages in **two formats**, text and CSV, as in the expected output.

Code rules

- Queries over the list are done with **collection functions**, not `for` / `while` loops. You may use a loop or `forEach` only for printing.
- Averages per year and month and the number of days per category are calculated by **grouping**, not with a separate query for each year, month or category.
- Printing a date in the format `27.12.2016.` is done by an **extension function** that you write yourself.
- The code that prints the yearly averages **must not know which format it uses**: it receives the format as a parameter. Adding a third format must not require changing that code.
- The same code processes both stations. You do not write separate functions for Skver and Lukavac.
- The `!!` operator is not allowed.

Expected output

Your program must print this (print averages with one decimal):

```
=== AIR QUALITY: TUZLA CANTON ===

--- Skver ---
Days loaded: 3483
Invalid lines: 0
Days without PM2.5: 201
Period: 27.12.2016. – 21.10.2026.
Worst day: 31.01.2017. (PM2.5 AQI 392)
Days per category (PM2.5):
  Good: 648
  Moderate: 1612
  Unhealthy for Sensitive Groups: 669
  Unhealthy: 281
  Very Unhealthy: 68
  Hazardous: 4
Most polluted month: January (141.0)
Cleanest month: May (52.6)

--- Lukavac ---
Days loaded: 3551
Invalid lines: 0
Days without PM2.5: 165
Period: 27.12.2016. – 22.10.2026.
Worst day: 01.02.2017. (PM2.5 AQI 450)
Days per category (PM2.5):
  Good: 652
  Moderate: 1669
  Unhealthy for Sensitive Groups: 708
  Unhealthy: 269
  Very Unhealthy: 79
  Hazardous: 9
Most polluted month: January (140.4)
Cleanest month: May (59.2)

Average PM2.5 AQI per year (text):
  2016: Skver 97.5 · Lukavac 109.8
  2017: Skver 96.6 · Lukavac 110.9
  2018: Skver 97.4 · Lukavac 110.0
  2019: Skver 91.4 · Lukavac 94.4
  2020: Skver 90.3 · Lukavac 95.1
  2021: Skver 88.3 · Lukavac 89.4
  2022: Skver 77.8 · Lukavac 85.5
  2023: Skver 77.7 · Lukavac 74.2
  2024: Skver 78.7 · Lukavac 69.0
```

```
  2025: Skver 89.4 · Lukavac 78.6
  2026: Skver 85.1 · Lukavac 69.5

Average PM2.5 AQI per year (CSV):
year;skver;lukavac
2016;97.5;109.8
2017;96.6;110.9
2018;97.4;110.0
2019;91.4;94.4
2020;90.3;95.1
2021;88.3;89.4
2022;77.8;85.5
2023;77.7;74.2
2024;78.7;69.0
2025;89.4;78.6
2026;85.1;69.5
```

Work and submission rules

- **No AI.** Do not use ChatGPT or similar tools, and turn off AI completion in your editor. You write the code in IntelliJ IDEA; how to set it up and turn off AI is described in `INSTRUCTIONS.md`.
- **Submission via Classmoji**, in the repository you receive (steps in `INSTRUCTIONS.md`). You need **at least 5 commits** that follow your progress, e.g. model, reading the files, queries, report formats. One big commit at the end leads to fewer points.
- **Live defense in the lab, without AI.** Justify your model, explain your code, then do a small additional task in your own code.

Grading

- The homework is passed if all model requirements and code rules are met, the output is correct and the defense is successful.
- **Fewer points for:** incorrect output, a model that does not meet the requirements, breaking the code rules, a weak defense.
- A successfully defended version C earns **+1 bonus activity point**.
- **If at the defense the student cannot show that they wrote the code themselves, the homework is worth 0 points.**